

WMM Laboratoria #5

Grupa poniedziałkowa

Sprawozdanie

Kacper Skrodzki

Oryginalne obrazy



Wprowadzenie

W ramach tych laboratoriów należało zaznajomić się z dalszymi aspektami obróbki obrazów – entropią i kompresją obrazów (koder PNG i JPEG). Ponadto trzeba było poddać analizie wpływ entropii na kompresję, postrzeganie barw i światła przez ludzkie oko oraz entropię w obrazach dla różnych przypadków.

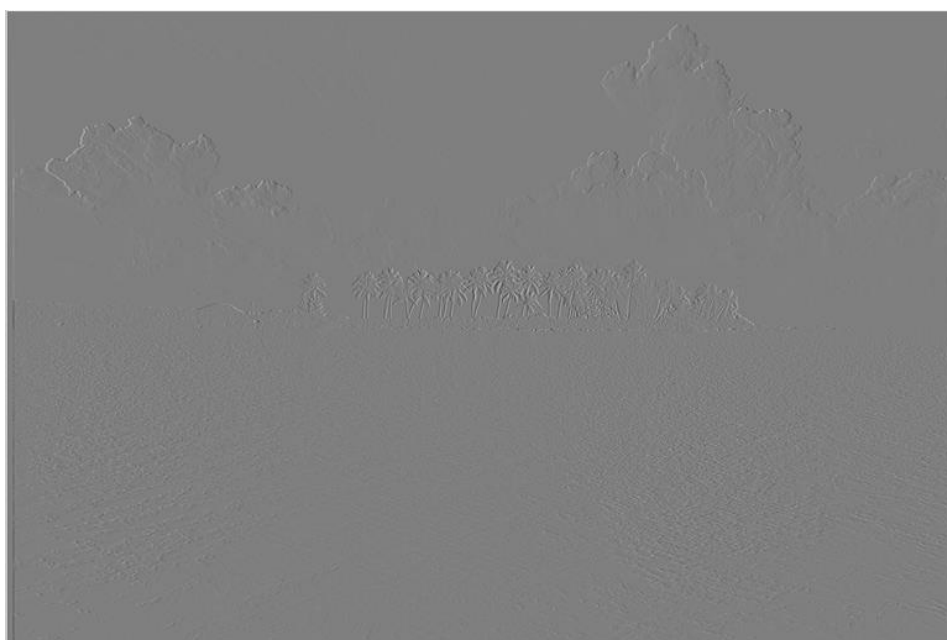
Część pierwsza – obraz monochromatyczny

Obliczanie entropii

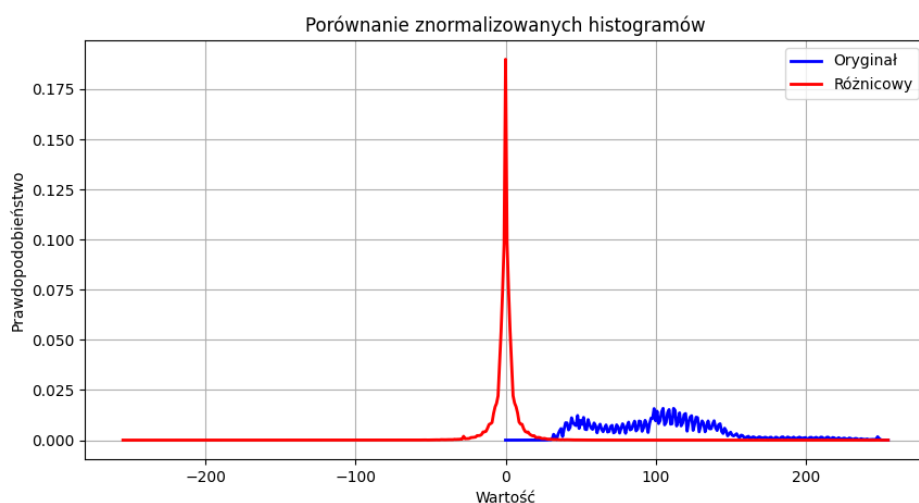
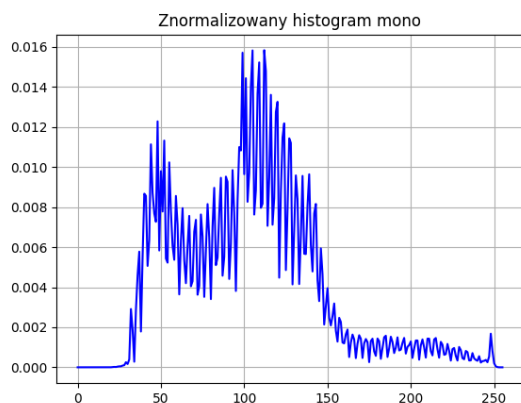
$$H(\text{obraz_mono}) = 7.2102$$

Dla obrazu monochromatycznego została wyznaczona entropia (*wynik powyżej*). Bacząc na to, że wynik znajduje się blisko górnej granicy (*8 bitów*) można uznać, że obraz jest wysokiej jakości, ostry i szczegółowy. Występują prawie wszystkie wartości pikseli z przedziału $[0, 255]$.

Obraz różnicowy



Powyższy obraz jest wynikiem predykcji horyzontalnej obrazu oryginalnego (obraz różnicowy). W wyniku tego działania, większość pikseli ma podobną wartość (zlewają się w jeden odcień szarości).



Entropia obrazu różnicowego:

$$H(\text{obraz_różnicowy}) = 4.4514$$

Na powyższych histogramach możemy zauważyć ewidentne różnice w rozkładzie pikseli między oryginałem, a obrazem różnicowym. W przypadku oryginału, piksele są rozłożone po całym przedziale $[0, 255]$ z różnym natężeniem. W przypadku obrazu różnicowego, prawie wszystkie piksele skupiają się wokół zera. Odchylenia stanowią odłamek całości i są powodowane przez ostre krawędzie na obrazie.

Takie pozycjonowanie pikseli skutkuje o wiele niższą entropią obrazu różnicowego względem obrazu oryginalnego (*około o 3 bity*). Dlaczego? Większość informacji znajduje się w bardzo wąskiej przestrzeni. Dzięki temu wiele pikseli można zakodować tym samym, krótkim kodem (*np. 01*) co zmniejsza dość mocno entropię. Dlatego obraz różnicowy jest wspaniały do kompresji, bo ma niską entropię.

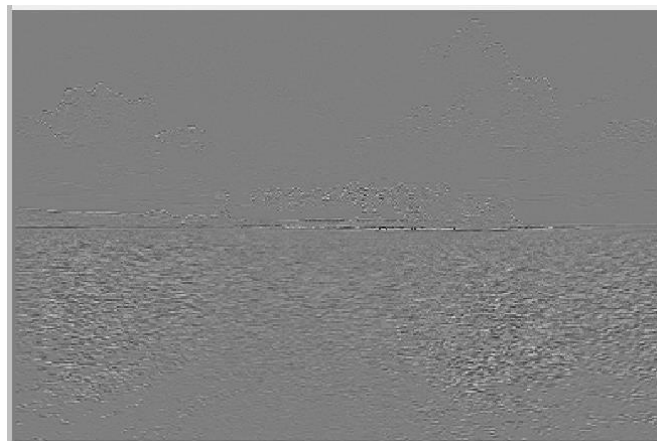
Współczynniki DWT

Wizualizacje pasm:

- **LL** – entropia $H(LL) = 7.2591$



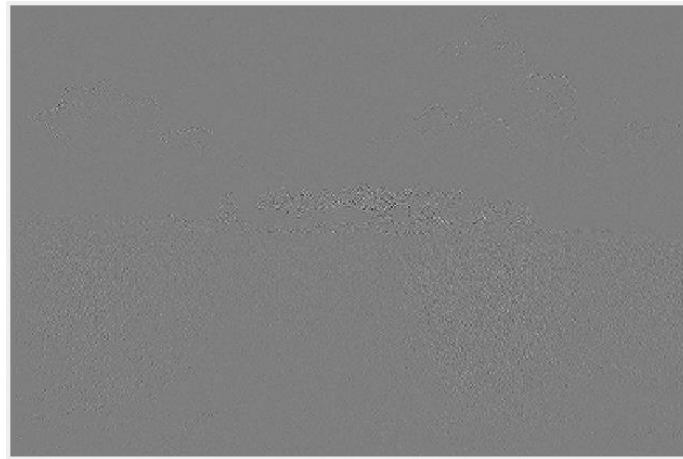
- **LH** - entropia $H(LH) = 5.0180$



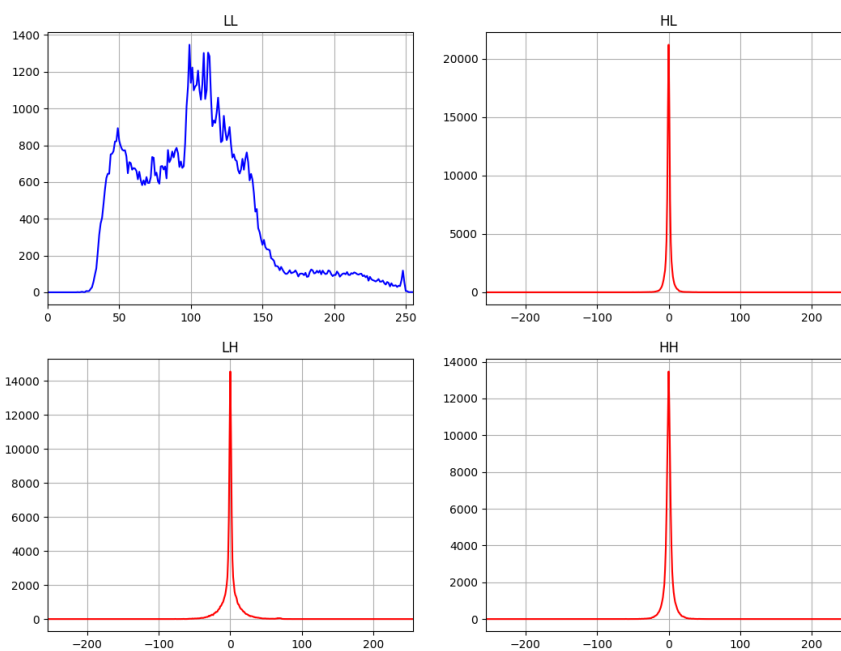
- **HL** – entropia $H(HL) = 3.7595$



- **HH** – entropia $H(HH) = 4.3650$



Histogramy poszczególnych pasm:



Z powyższych histogramów i entropii poszczególnych pasm DWT wynika parę wniosków. Po pierwsze pasmo LL ma bardzo podobną entropię i histogram do oryginalnego obrazu. Sama wizualizacja tego pasma jest po prostu rozmytym oryginałem. Jediną rzucającą się w oczy różnicą jest sam przebieg histogramu – jest on gładki w porównaniu do oryginału.

Co da pozostałych pasm – LH, HL, HH. Wyglądem i entropią są bliskie obrazu różnicowego. Oscylują wokół entropi na poziomie 3 – 5 bitów. Ich histogramy są o wiele gładzsze niż obraz różnicowy – pik w 0.

Takie wyniki zgadzają się z rzeczywistością. Pasma LL zawiera większość elementów obrazu, gdzie pasma LH, HL i HH wykrywają krawędzie poziome, pionowe i ukośne. W przypadku kompresji to oznacza, że można zignorować dokładność krawędzi i je mocno skompresować.

Przeptywność

Bitrate: 4.6006

```
H(obraz_mono) = 7.2102
H(obraz_różnicowy) = 4.4514
H(LL) = 7.2591
H(LH) = 5.0180
H(HL) = 3.7595
H(HH) = 4.3650
H_śr = 5.1004
```

Średnia bitowa (bitrate) obrazu zakodowanym kodekiem PNG jest zbliżona do entropi obrazu różnicowego. Jest znacznie mniejsza od entropi pasma LL. W przypadku HL i HH jest większa, a w stosunku do LH jest mniejsza. Przeptywność mniejsza od entropii nie przeczy twierdzeniu $l_{sr} \geq H$, ponieważ entropia zakłada niezależność pikseli od siebie nawzajem. Kodek PNG bazuje na predykcji oryginalnego obrazu (obraz różnicowy), aby dokonać kompresji. Zatem, zasada bezstratnej kompresji nie została złamana – trzeba porównać bitrate z entropią obrazu różnicowego, aby potwierdzić tę zasadę (*a jak widać powyżej, to się wszystko zgadza: $4.6006 \geq 4.4514$*).

Część druga – obraz barwny

Składowe RGB i YCrCb

Entropie poszczególnych kanałów:

- Kanał R – entropia $H(R) = 7.2197$



- Kanał G – entropia $H(G) = 7.3047$



- Kanał B – entropia $H(B) = 7.1979$



- Kanał Y – entropia $H(Y) = 7.1856$



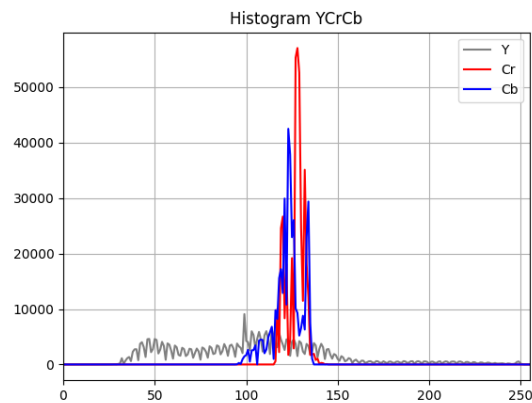
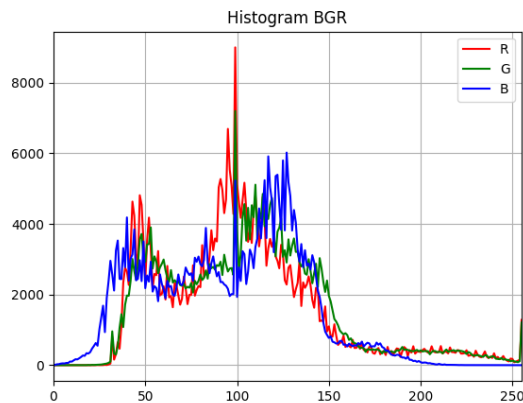
- Kanał Cr – entropia $H(\text{Cr}) = 3.8282$



- Kanał Cb – entropia $H(\text{Cb}) = 4.6093$



Histogramy dla RGB i YCrCb

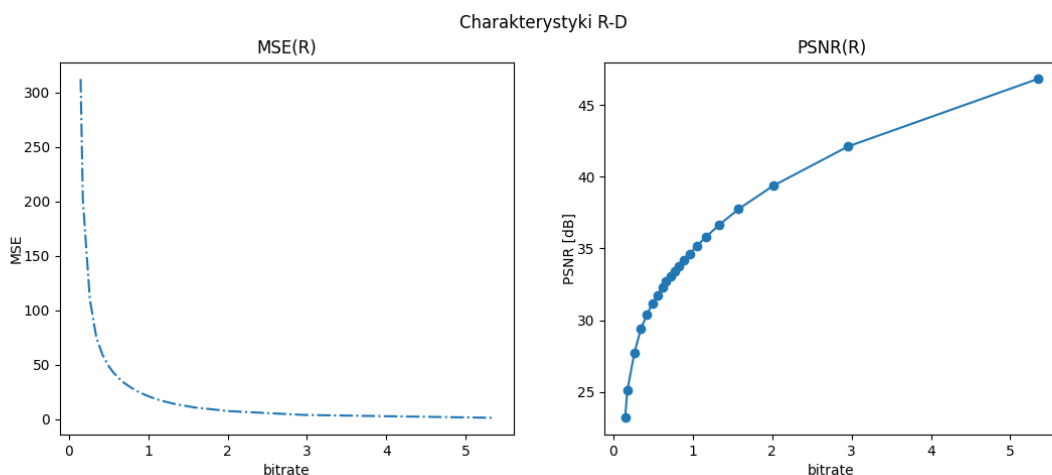


```
H(R) = 7.2197 H(Y) = 7.1856  
H(G) = 7.3047 H(Cr) = 3.8282  
H(B) = 7.1979 H(Cb) = 4.6093  
H_sr = 7.2408 H_sr = 5.2077
```

Z powyższych obrazków możemy wyciągnąć parę wniosków. Po pierwsze, entropia w kanałach RGB jest w miarę równo rozłożona – bliska wartości oryginalnego obrazu. Za to po konwersji na kanały YCrCb nastąpiła spora zmiana. Kanał Y, czyli luminancja, ma bardzo wysoką entropię – bliską wartości kanałów RGB i oryginału. Za to chrominancja – Cr i Cb – mają bardzo niskie wartości, podobne do wartości dla pasm HL i HH w DWT.

Fakt ten jest dość pomocny w kompresji. Sama struktura Cr i Cb ułatwia kodowanie dla kodeków oraz ludzkie oko jest bardziej czułe na luminancję (*światło*). Z tego powodu, można dość silnie zniekształcić kanały Cr i Cb bez wpływu na odbiór treści przez człowieka – nie dojrzy różnicy, bo to kanał Y niesie najistotniejsze informacje o obrazie.

Zależność zniekształceń od przepływności



Subiektywna ocena obrazów po kompresji kodekiem JPEG.

- Jakość **bardzo zła** – quality 1-5



- Jakość **zła** – quality 10-15



- Jakość **słaba** – quality 20-35



- Jakość **średnia** – quality 40-50



- Jakość **dobra** – quality 55-65



- Jakość **bardzo dobra** – quality 70-85



- Jakość **wysmienita** – quality 90-99



Wynikowe wartości bitrate-u, MSE i PSNR dla każdego quality

```
Quality= 1: Bitrate=0.1494, MSE=312.4747, PSNR=23.1827 dB
Quality= 5: Bitrate=0.1775, MSE=202.0526, PSNR=25.0762 dB
Quality=10: Bitrate=0.2657, MSE=110.1627, PSNR=27.7105 dB
Quality=15: Bitrate=0.3484, MSE=74.7520, PSNR=29.3946 dB
Quality=20: Bitrate=0.4260, MSE=59.0841, PSNR=30.4161 dB
Quality=25: Bitrate=0.4948, MSE=49.7457, PSNR=31.1632 dB
Quality=30: Bitrate=0.5574, MSE=43.3678, PSNR=31.7591 dB
Quality=35: Bitrate=0.6185, MSE=38.6219, PSNR=32.2625 dB
Quality=40: Bitrate=0.6670, MSE=35.0838, PSNR=32.6797 dB
Quality=45: Bitrate=0.7232, MSE=32.1077, PSNR=33.0647 dB
Quality=50: Bitrate=0.7734, MSE=29.6287, PSNR=33.4137 dB
Quality=55: Bitrate=0.8214, MSE=27.3935, PSNR=33.7543 dB
Quality=60: Bitrate=0.8832, MSE=24.9787, PSNR=34.1551 dB
Quality=65: Bitrate=0.9595, MSE=22.4980, PSNR=34.6094 dB
Quality=70: Bitrate=1.0555, MSE=19.6891, PSNR=35.1885 dB
Quality=75: Bitrate=1.1632, MSE=17.1488, PSNR=35.7885 dB
Quality=80: Bitrate=1.3324, MSE=14.1048, PSNR=36.6371 dB
Quality=85: Bitrate=1.5793, MSE=10.8983, PSNR=37.7572 dB
Quality=90: Bitrate=2.0127, MSE=7.5162, PSNR=39.3708 dB
Quality=95: Bitrate=2.9581, MSE=3.9851, PSNR=42.1264 dB
Quality=99: Bitrate=5.3429, MSE=1.3497, PSNR=46.8284 dB
```

Przepływność dla obrazu barwnego PNG

Bitrate dla PNG: 11.6773

Format	Przepływność [bit/pixel]
PNG (monochromatyczny)	4.6006
PNG (barwny)	11.6773
JPEG Q=5 (barwny, bardzo zły)	0.1775
JPEG Q=15 (barwny, zły)	0.3484
JPEG Q=35 (barwny, słaby)	0.6185
JPEG Q=50 (barwny, średni)	0.7734
JPEG Q=65 (barwny, dobry)	0.9585
JPEG Q=85 (barwny, bardzo dobry)	1.5793
JPEG Q=99 (barwny, wyśmienity)	5.3429

Na powyższych obrazach, wykresach, zdjęciach i tabelce jest ukazany proces testowania jakości kompresji za pomocą formatu JPEG. Podsumowując, można zauważyć następujące fakty:

- Przepływność dla barwnego PNG jest o wiele większa od monochromatycznego PNG – ma to sens, gdyż zamiast jednego kanału są trzy kanały do kompresji.
- Większość uzyskanych obrazów cechuje się zauważalnymi stratami w jakości obrazu. O tym fakcie świadczy wartość przepływności dla wielu kategorii (*zakres $[0, 1]$*) oraz wykres zniekształceń w stosunku do przepływności. Jest to znak kompresji stratnej, gdyż została złamana zasada $I_{sr} \geq H$. Dla jakości „wyśmienitej”, przepływność osiąga bezstratną kompresję.
- Jednakże bezstratna kompresja nie jest do końca zła. Patrzymy na kategorię „dobry” i „bardzo dobry” – subiektywnie obrazy wyglądają normalnie. Tutaj straty są prawie niezauważalne.
- JPEG jest bardzo dobry w kompresji, która zachowuje dobrą jakość, ale kosztem możliwej straty danych. PNG za to jest całkowicie bezstratny, lecz ograniczony w stopniu kompresji. Różnicę tą można zauważyć w różnicy przepływności między PNG a JPEG ($11.6773 \gg 5.3429$).

Kod źródłowy

```
import os

import cv2

import numpy as np

import matplotlib.pyplot as plt

""" ===== FUNKCJE POMOCNICZE ===== """

def printi(img, img_title="image"):

    """Pomocnicza funkcja do wypisania informacji o obrazie."""

    print(f"{img_title}, wymiary: {img.shape}, typ danych: {img.dtype}, wartości: {img.min()} - {img.max()}")

def cv_imshow(img, img_title="image"):

    """

    Funkcja do wyświetlania obrazu w wykorzystaniem okna OpenCV.

    Wykonywane jest przeskalowanie obrazu z rzeczywistymi lub 16-bitowymi całkowitoliczbowymi wartościami pikseli,

    żeby jedną funkcją wywietlać obrazy różnych typów.

    """

    if (img.dtype == np.float32) or (img.dtype == np.float64):

        img_ = img / 255

    elif img.dtype == np.int16:

        img_ = img * 128

    else:

        img_ = img

    cv2.imshow(img_title, img_)

    cv2.waitKey(0)
```



```
"""
```

Obliczanie entropii

```
"""
```

```
def calc_entropy(hist):
```

```
    pdf = (
```

```
        hist / hist.sum()
```

```
    ) # normalizacja histogramu -> rozkład prawdopodobieństwa; UWAGA: niebezpieczeństwo '/0' dla 'zerowego' histogramu!!!
```

```
    # entropy = -(pdf*np.log2(pdf)).sum() ### zapis na tablicach, ale problem z '/0'
```

```
    entropy = -sum([x * np.log2(x) for x in pdf if x != 0])
```

```
    return entropy
```

```
"""
```

Transformacja falkowa obrazu

```
"""
```

```
def dwt(img):
```

```
    """
```

Bardzo prosta i podstawowa implementacja, nie uwzględniająca efektywnych metod obliczania DWT

i dopuszczająca pewne niedokładności.

```
    """
```

```
    maskL = np.array(
```

```
        [
```

```
            0.02674875741080976,
```

```
            -0.01686411844287795,
```

```
            -0.07822326652898785,
```

```
            0.2668641184428723,
```

```
            0.6029490182363579,
```

```
            0.2668641184428723,
```

```
            -0.07822326652898785,
```

```

        -0.01686411844287795,
        0.02674875741080976,
    ]
)
maskH = np.array(
    [
        0.09127176311424948,
        -0.05754352622849957,
        -0.5912717631142470,
        1.115087052456994,
        -0.5912717631142470,
        -0.05754352622849957,
        0.09127176311424948,
    ]
)

bandLL = cv2.sepFilter2D(img, -1, maskL, maskL)[::2, ::2]
bandLH = cv2.sepFilter2D(img, cv2.CV_16S, maskL, maskH)[::2, ::2]
# ze względu na filtrację górnoprzepustową -> wartości ujemne, dlatego wynik 16-bitowy ze
znakiem
bandHL = cv2.sepFilter2D(img, cv2.CV_16S, maskH, maskL)[::2, ::2]
bandHH = cv2.sepFilter2D(img, cv2.CV_16S, maskH, maskH)[::2, ::2]

return bandLL, bandLH, bandHL, bandHH

""" Wyznaczanie charakterystyki R-D """

def calc_mse_psnr(img1, img2):
    """Funkcja obliczająca MSE i PSNR dla różnicy podanych obrazów, zakładana wartość pikseli z
    przedziału [0, 255]."""

    imax = 255.0**2 # maksymalna wartość sygnału -> 255

```

```
"""
```

W różnicy obrazów istotne są wartości ujemne, dlatego img1 konwertowany jest do typu np.float64 (liczby rzeczywiste)

aby nie ograniczać wyniku do przedziału [0, 255].

```
"""
```

```
mse = (  
    ((img1.astype(np.float64) - img2) ** 2).sum() / img1.size  
)  
psnr = 10.0 * np.log10(imax / mse)  
return (mse, psnr)
```

```
""" GŁÓWNY PROGRAM """
```

```
""" OBRAZ MONOCHROMATYCZNY """
```

```
images_dir = "images/"
```

```
def main():
```

```
    image_mono = cv2.imread(images_dir + "wyspa_mono.png", cv2.IMREAD_UNCHANGED)
```

```
    printi(image_mono)
```

```
    bitrate_mono = 8 * os.stat(images_dir + "wyspa_mono.png").st_size / (image_mono.shape[0] *  
image_mono.shape[1])
```

```
    hist_mono_img = cv2.calcHist([image_mono], [0], None, [256], [0, 256]).flatten()
```

```
    hist_mono_norm = hist_mono_img / hist_mono_img.sum()
```

```
    entropy_mono = calc_entropy(hist_mono_img)
```

```
""" Obraz różnicowy """
```

```
img_tmp1 = image_mono[:, 1:] # wszystkie wiersze (':'), kolumny od 'pierwszej' do ostatniej ('1:')
```

```
img_tmp2 = image_mono[:, :-1] # wszystkie wiersze, kolumny od 'zerowej' do przedostatniej (':-1')
```

```
image_hdiff = cv2.addWeighted(img_tmp1, 1, img_tmp2, -1, 0, dtype=cv2.CV_16S)
```

```
printi(image_hdiff, "image_hdiff")
```

```
image_hdiff_0 = cv2.addWeighted(image_mono[:, 0], 1, 0, 0, -127, dtype=cv2.CV_16S)
```

```
# od 'zerowej' kolumny obrazu oryginalnego odejmowana stała wartość '127'
```

```
printi(image_hdiff_0, "image_hdiff_0")
```

```
image_hdiff = np.hstack((image_hdiff_0, image_hdiff)) # połączenie tablic w kierunku poziomym, czyli 'kolumna za kolumną'
```

```
printi(image_hdiff, "image_hdiff")
```

```
cv_imshow(image_hdiff, "image_hdiff")
```

```
hdiff_tmp = (image_hdiff + 255).astype(np.uint16)
```

```
hist_hdiff = cv2.calcHist([hdiff_tmp], [0], None, [511], [0, 511]).flatten()
```

```
hist_hdiff_norm = hist_hdiff / hist_hdiff.sum()
```

```
entropy_hdiff = calc_entropy(hist_hdiff)
```

```
""" Wyświetlenie histogramów oryginału i różnicowego """
```

```
x1 = np.arange(256)
```

```
x2 = np.arange(-255, 256, 1)
```

```
plt.figure()
```

```
plt.plot(x1, hist_mono_norm, color="blue")
```

```
plt.title("Znormalizowany histogram mono")
```

```
plt.grid(True)
```

```
plt.savefig("hist_mono.png")
```

```
plt.figure()
```

```
plt.plot(x2, hist_hdiff_norm, color="red")
```

```
plt.title("Znormalizowany histogram różnicowy")
```

```
plt.grid(True)
```

```
plt.savefig("hist_hdiff.png")
```

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(x1, hist_mono_norm, color="blue", label="Oryginał", linewidth=2)
```

```
plt.plot(x2, hist_hdiff_norm, color="red", label="Różnicowy", linewidth=2)
```

```
plt.title("Porównanie znormalizowanych histogramów")
```

```
plt.xlabel("Wartość")
```

```
plt.ylabel("Prawdopodobieństwo")
```

```
plt.grid(True)
```

```
plt.legend()
```

```
plt.savefig("compare_hist_mono.png")
```

```
plt.show()
```

```
""" Wylizanie współczynników DWT """
```

```
ll, lh, hl, hh = dwt(image_mono)
```

```
printi(ll, "LL")
```

```
printi(lh, "LH")
```

```
printi(hl, "HL")
```

```
printi(hh, "HH")
```

```
cv_imshow(ll, "LL2")
```

```

cv_imshow(cv2.multiply(lh, 2), "LH2")
cv_imshow(cv2.multiply(hl, 2), "HL2")
cv_imshow(cv2.multiply(hh, 2), "HH2")

hist_ll = cv2.calcHist([ll], [0], None, [256], [0, 256]).flatten()
hist_lh = cv2.calcHist([(lh + 255).astype(np.uint16)], [0], None, [511], [0, 511]).flatten()
hist_hl = cv2.calcHist([(hl + 255).astype(np.uint16)], [0], None, [511], [0, 511]).flatten()
hist_hh = cv2.calcHist([(hh + 255).astype(np.uint16)], [0], None, [511], [0, 511]).flatten()

H_ll = calc_entropy(hist_ll)
H_lh = calc_entropy(hist_lh)
H_hl = calc_entropy(hist_hl)
H_hh = calc_entropy(hist_hh)

print(f"Bitrate: {bitrate_mono:.4f}")
print(f"H(obraz_mono) = {entropy_mono:.4f}")
print(f"H(obraz_różnicowy) = {entropy_hdiff:.4f}")

print(f"H(LL) = {H_ll:.4f} \nH(LH) = {H_lh:.4f} \nH(HL) = {H_hl:.4f} \nH(HH) = {H_hh:.4f} \nH_śr = {(H_ll+H_lh+H_hl+H_hh)/4:.4f}")

fig = plt.figure()
fig.set_figheight(fig.get_figheight() * 2) # zwiększenie rozmiarów okna
fig.set_figwidth(fig.get_figwidth() * 2)
plt.subplot(2, 2, 1)
plt.plot(hist_ll, color="blue")
plt.title("LL")
plt.xlim([0, 255])
plt.grid(True)
plt.subplot(2, 2, 3)
plt.plot(np.arange(-255, 256, 1), hist_lh, color="red")
plt.title("LH")
plt.xlim([-255, 255])

```



```
plt.grid(True)
plt.subplot(2, 2, 2)
plt.plot(np.arange(-255, 256, 1), hist_hl, color="red")
plt.title("HL")
plt.xlim([-255, 255])
plt.grid(True)
plt.subplot(2, 2, 4)
plt.plot(np.arange(-255, 256, 1), hist_hh, color="red")
plt.title("HH")
plt.xlim([-255, 255])
plt.grid(True)
```

```
plt.savefig("compare_hist_DWT.png")
plt.close()
```

"""" OBRAZ BARWNY """"

```
image_col = cv2.imread(images_dir + "wyspa_col.png")
printi(image_col)
```

cv2.imread wczytuje obraz kolorowy domyślnie jako BGR

```
image_r = image_col[:, :, 2]
image_g = image_col[:, :, 1]
image_b = image_col[:, :, 0]
```

```
hist_r = cv2.calcHist([image_r], [0], None, [256], [0, 256]).flatten()
hist_g = cv2.calcHist([image_g], [0], None, [256], [0, 256]).flatten()
hist_b = cv2.calcHist([image_b], [0], None, [256], [0, 256]).flatten()
```

```
H_R = calc_entropy(hist_r)
H_G = calc_entropy(hist_g)
```

```

H_B = calc_entropy(hist_b)

print(f"H(R) = {H_R:.4f} \nH(G) = {H_G:.4f} \nH(B) = {H_B:.4f} \nH_śr = {(H_R+H_G+H_B)/3:.4f}")


cv_imshow(image_r, "R")
cv_imshow(image_g, "G")
cv_imshow(image_b, "B")

plt.figure()

plt.plot(hist_r, color="red", label="R")
plt.plot(hist_g, color="green", label="G")
plt.plot(hist_b, color="blue", label="B")

plt.title("Histogram RGB")

plt.xlim([0, 255])

plt.grid(True)

plt.legend()


plt.savefig("hist_bgr.png")

plt.show()


image_YCrCb = cv2.cvtColor(image_col, cv2.COLOR_BGR2YCrCb)

printi(image_YCrCb, "Image_YCrCb")


hist_y = cv2.calcHist([image_YCrCb[:, :, 0]], [0], None, [256], [0, 256]).flatten()
hist_cr = cv2.calcHist([image_YCrCb[:, :, 1]], [0], None, [256], [0, 256]).flatten()
hist_cb = cv2.calcHist([image_YCrCb[:, :, 2]], [0], None, [256], [0, 256]).flatten()


H_Y = calc_entropy(hist_y)
H_Cr = calc_entropy(hist_cr)
H_Cb = calc_entropy(hist_cb)

print(f"H(Y) = {H_Y:.4f} \nH(Cr) = {H_Cr:.4f} \nH(Cb) = {H_Cb:.4f} \nH_śr = {(H_Y+H_Cr+H_Cb)/3:.4f}")

```

```

cv_imshow(image_YCrCb[:, :, 0], "Y")
cv_imshow(image_YCrCb[:, :, 1], "Cr")
cv_imshow(image_YCrCb[:, :, 2], "Cb")
plt.figure()
plt.plot(hist_y, color="gray", label="Y")
plt.plot(hist_cr, color="red", label="Cr")
plt.plot(hist_cb, color="blue", label="Cb")
plt.title("Histogram YCrCb")
plt.xlim([0, 255])
plt.grid(True)
plt.legend()

plt.savefig("hist_ycrcb.png")
plt.show()

xx = []
ym = []
yp = []

for quality in [1, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95, 99]:
    out_file_name = f"out_image_q{quality:03d}.jpg"
    """ Zapis do pliku w formacie .jpg z ustaloną 'jakością' """
    cv2.imwrite(out_file_name, image_col, (cv2.IMWRITE_JPEG_QUALITY, quality))
    """ Odczyt skompresowanego obrazu, policzenie bitrate'u i PSNR """
    image_compressed = cv2.imread(out_file_name, cv2.IMREAD_UNCHANGED)
    bitrate = 8 * os.stat(out_file_name).st_size / (image_col.shape[0] * image_col.shape[1]) ###
    image.shape == image_compressed.shape

    mse, psnr = calc_mse_psnr(image_col, image_compressed)

xx.append(bitrate)

```

```
ym.append(mse)
```

```
yp.append(psnr)
```

```
""" Logi na terminal do łatwego podglądu """
```

```
print(f"Quality={quality:2d}: Bitrate={bitrate:.4f}, MSE={mse:.4f}, PSNR={psnr:.4f} dB")
```

```
fig = plt.figure()
```

```
fig.set_figwidth(fig.get_figwidth() * 2)
```

```
plt.suptitle("Charakterystyki R-D")
```

```
plt.subplot(1, 2, 1)
```

```
plt.plot(xx, ym, "-.")
```

```
plt.title("MSE(R)")
```

```
plt.xlabel("bitrate")
```

```
plt.ylabel("MSE", labelpad=0)
```

```
plt.subplot(1, 2, 2)
```

```
plt.plot(xx, yp, "-o")
```

```
plt.title("PSNR(R)")
```

```
plt.xlabel("bitrate")
```

```
plt.ylabel("PSNR [dB]", labelpad=0)
```

```
plt.savefig("r-d_characteristics")
```

```
plt.show()
```

```
bitrate_png = 8 * os.stat(images_dir + "wyspa_col.png").st_size / (image_mono.shape[0] *  
image_mono.shape[1])
```

```
print(f"Bitrate dla PNG: {bitrate_png:.4f}")
```

```
if __name__ == "__main__":
```

```
    main()
```